

Лабораторная №1

CRUD на Spring Boot, Junit, Lombok, Mapstruct

Типовая задача в веб-разработке — это создание API для CRUD сущности.

CRUD (Create/Read/Update/Delete) — это набор типовых операций, который обычно выполняется над сущностью. В этом уроке мы соберем все изученное и создадим эталонный CRUD на примере поста в блог.

Общий план создания CRUD выглядит так:

- . Создаем сущность
- . Создаем репозиторий
- . Создаем DTO
- . Создаем маппер
- . Реализуем контроллер (логику, которая реализуется в примере в контроллере необходимо вынести в сервис)
- . Пишем интеграционные тесты

Создание сущности

У поста в блоге есть четыре основных свойства:

Название
Слаг
Текст
Автор

Кроме того, еще можно добавить дату создания и последнего обновления:

```
package io.hexlet.spring.model;
```

```
import static jakarta.persistence.GenerationType.IDENTITY;

import java.time.LocalDate;

import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedDate; import
org.springframework.data.jpa.domain.support.AuditingEntityListener;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.EntityListeners;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.EqualsAndHashCode;
import lombok.Getter;
import lombok.Setter;
import lombok.ToString;

@Entity
@Getter
@Setter
@EntityListeners(AuditingEntityListener.class)
@ToString(includeFieldNames = true, onlyExplicitlyIncluded = true)
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
@Table(name = "posts")
// Позже мы обсудим BaseEntity подробнее
public class Post implements BaseEntity {
    @Id
    @GeneratedValue(strategy = IDENTITY)
    @ToString.Include
```

```
@EqualsAndHashCode.Include
private Long id;

@ManyToOne
@NotNull
private User author;

@Column(unique = true)
@ToString.Include
@NotNull
private String slug;

@NotBlank
@ToString.Include
private String name;

@NotBlank
@ToString.Include
@Column(columnDefinition = "TEXT")
private String body;

@LastModifiedDate
private LocalDate updatedAt;

@CreatedDate
private LocalDate createdAt;
}
```

Создание репозитория

Использование слага в URL-адресе подразумевает, что мы сможем делать выборку сущности по слягу:

```
postRepository.findBySlug(/* slug */);
```

Сразу добавим этот метод в репозиторий:

```
package io.hexlet.spring.repository;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import io.hexlet.spring.model.Post;

@Repository
public interface PostRepository extends JpaRepository<Post, Long> {
    Optional<Post> findBySlug(String slug);
}
```

Создание DTO

В целом нам понадобятся три разных DTO — для создания, обновления и просмотра поста:

```
// PostCreateDTO

package io.hexlet.spring.dto;

import jakarta.validation.constraints.NotNull;
import lombok.Getter;
import lombok.Setter;

@Setter
@Getter
public class PostCreateDTO {
    @NotNull
```

```
    private Long authorId;

    @NotNull
    private String slug;

    @NotNull
    private String name;

    @NotNull
    private String body;
}
```

// PostDTO

```
package io.hexlet.spring.dto;
```

```
import java.time.LocalDate;
```

```
import lombok.Getter;
```

```
import lombok.Setter;
```

```
@Setter
```

```
@Getter
```

```
public class PostDTO {
    private Long id;
    private Long authorId;
    private String slug;
    private String name;
    private String body;
    private LocalDate createdAt;
}
```

// PostUpdateDTO

```
package io.hexlet.spring.dto;
```

```
import org.openapitools.jackson.nullable.JsonNullable;
import jakarta.validation.constraints.NotNull;
import lombok.Getter;
import lombok.Setter;

@Setter
@Getter
public class PostUpdatedDTO {
    @NotNull
    private JsonNullable<Long> authorId;

    @NotNull
    private JsonNullable<String> slug;

    @NotNull
    private JsonNullable<String> name;

    @NotNull
    private JsonNullable<String> body;
}
```

Создание маппера

Для CRUD нам нужны три операции:

Конвертация DTO для создания в пост

Конвертация поста в DTO для просмотра

Обновление поста на основе DTO для обновления

```
package io.hexlet.spring.mapper;

import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
```

```

import org.mapstruct.MappingConstants;
import org.mapstruct.MappingTarget;
import org.mapstruct.NullValuePropertyMappingStrategy;
import org.mapstruct.ReportingPolicy;

import io.hexlet.spring.dto.PostCreatedDTO;
import io.hexlet.spring.dto.PostDTO;
import io.hexlet.spring.dto.PostUpdatedDTO;
import io.hexlet.spring.model.Post;

@Mapper(
    uses = { JsonNullableMapper.class, ReferenceMapper.class },
    nullValuePropertyMappingStrategy = NullValuePropertyMappingStrategy.IGNORE,
    componentModel = MappingConstants.ComponentModel.SPRING,
    unmappedTargetPolicy = ReportingPolicy.IGNORE
)
public abstract class PostMapper {
    @Mapping(target = "author", source = "authorId")
    public abstract Post map(PostCreatedDTO dto);

    @Mapping(source = "author.id", target = "authorId")
    public abstract PostDTO map(Post model);

    @Mapping(target = "author", source = "authorId")
    public abstract void update(PostUpdatedDTO dto, @MappingTarget Post model);
}

```

Самое интересное в этом коде — это конвертация `authorId` из DTO в свойство `author` внутри поста. Чтобы выполнить эту операцию, нужно сделать запрос в базу данных и извлечь объект автора. По умолчанию MapStruct такого не умеет, но для него можно создать маппер, который решает эту задачу. Здесь мы сразу приведем его код:

```

package io.hexlet.spring.mapper;

import org.mapstruct.Mapper;

```

```

import org.mapstruct.MappingConstants;
import org.mapstruct.TargetType;
import org.springframework.beans.factory.annotation.Autowired;
import io.hexlet.spring.model.BaseEntity;
import jakarta.persistence.EntityManager;

@Mapper(
    componentModel = MappingConstants.ComponentModel.SPRING
)
public abstract class ReferenceMapper {
    @Autowired
    private EntityManager entityManager;

    public <T extends BaseEntity> T toEntity(Long id, @TargetType Class<T> entityClass) {
        return id != null ? entityManager.find(entityClass, id) : null;
    }
}

```

Чтобы этот код заработал, необходимо внедрить общий базовый интерфейс для всех моделей, на который маппер мог бы ориентироваться и понимать, применять метод `toEntity()`. Мы назовем его `BaseEntity`:

```

package io.hexlet.blog.model;

// По желанию его можно заполнять
// С точки зрения маппера важно только его наличие

public interface BaseEntity {
}

```

Определение класса `Post` в таком случае выглядит так:

```

public class Post implements BaseEntity {
    // Код класса
}

```


Создание теста

Код интеграционных тестов не завязан на устройство контроллера. Поэтому сам тест можно написать до реализации контроллера — так мы упростим создание контроллера и проверку его работоспособности. Такой подход называется TDD (Test Driven Development):

```
package io.hexlet.spring.controller.api;

import static net.javacrumbs.jsonunit.assertj.JsonAssertions.assertThatJson;
import static org.assertj.core.api.Assertions.assertThat;
import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.put;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import org.instancio.Instancio;
import org.instancio.Model;
import org.instancio.Select;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openapitools.jackson.nullable.JsonNullable;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.csrf;
import org.springframework.test.web.servlet.MockMvc;

import com.fasterxml.jackson.databind.ObjectMapper;
```

```
import io.hexlet.blog.model.Post;
import io.hexlet.blog.model.User;

import io.hexlet.spring.dto.PostUpdateDTO;
import io.hexlet.spring.mapper.PostMapper;
import io.hexlet.spring.repository.PostRepository;
import io.hexlet.spring.util.ModelGenerator;
import net.datafaker.Faker;
```

```
@SpringBootTest
```

```
@AutoConfigureMockMvc
```

```
public class PostsControllerTest {
```

```
    @Autowired
```

```
    private MockMvc mockMvc;
```

```
    @Autowired
```

```
    private ObjectMapper om;
```

```
    @Autowired
```

```
    private PostMapper postMapper;
```

```
    @Autowired
```

```
    private PostRepository postRepository;
```

```
    // Нужно создать бин
```

```
    @Autowired
```

```
    private Faker faker;
```

```
    private User testUser;
```

```
    private Post testPost;
```

```
    @BeforeEach
```

```
    public void setUp() {
```

```
        testUser = Instance.of(User.class)
```

```

        .ignore(Select.field(Post::getId))
        .supply(Select.field(User::getEmail), () -> faker.internet().emailAddress()
        .create());
testPost = Instancio.of(Post.class)
    .ignore(Select.field(Post::getId))
    .supply(Select.field(Post::getName), () -> faker.gameOfThrones().house())
    .supply(Select.field(Post::getBody), () -> faker.gameOfThrones().quote())
    .supply(Select.field(Post::getAuthor), testUser)
    .create();
}

```

@Test

```

public void testIndex() throws Exception {
    postRepository.save(testPost);

    var result = mockMvc.perform(get("/api/posts")
        .andExpect(status())
        .isOk())
        .andReturn();
    var body = result.getResponse().getContentAsString();
    assertThatJson(body).isArray();
}

```

@Test

```

public void testCreate() throws Exception {
    var dto = postMapper.map(testPost);

    var request = post("/api/posts")
        .contentType(MediaType.APPLICATION_JSON)
        .content(om.writeValueAsString(dto));

    mockMvc.perform(request)
        .andExpect(status().isCreated());

    var post = postRepository.findBySlug(dto.getSlug()).get();
    assertNotNull(post);
}

```

```

        assertThat(post.getName()).isEqualTo(dto.getName());
    }
    @Test
    public void testUpdate() throws Exception {
        postRepository.save(testPost);

        var dto = new PostUpdatedDTO();
        dto.setName(JsonNullable.of("new name"));

        var request = put("/api/posts/" + testPost.getId())
            .contentType(MediaType.APPLICATION_JSON)
            .content(om.writeValueAsString(dto));

        mockMvc.perform(request)
            .andExpect(status().isOk());

        post = postRepository.findById(testPost.getId()).get();
        assertThat(post.getName()).isEqualTo(dto.getName()); }

```

```

@Test
public void testShow() throws Exception {
    postRepository.save(testPost);

    var request = get("/api/posts/" + testPost.getId());
    var result = mockMvc.perform(request)
        .andExpect(status().isOk())
        .andReturn();
    var body = result.getResponse().getContentAsString();
    assertThatJson(body).and(
        v -> v.node("slug").isEqualTo(testPost.getSlug()),
        v -> v.node("name").isEqualTo(testPost.getName()), v
        -> v.node("body").isEqualTo(testPost.getBody()) );
}

```

```

@Test
public void testDestroy() throws Exception {

```

```

        postRepository.save(testPost);
        var request = delete("/api/posts/" + testPost.getId());
        mockMvc.perform(request)
                .andExpect(status().isNoContent());

        assertThat(postRepository.existsById(testPost.getId())).isEqualTo(false);
    }
}

```

Реализация контроллера

Перейдем к контроллеру. Здесь мы не добавляем ничего нового. Весь его код мы видели частями, а теперь собираем все вместе:

```

package io.hexlet.spring.controller.api;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import io.hexlet.spring.dto.PostCreatedDTO;
import io.hexlet.spring.dto.PostDTO;

```

```
import io.hexlet.spring.dto.PostUpdatedTO;
import io.hexlet.spring.exception.ResourceNotFoundException;
import io.hexlet.spring.mapper.PostMapper;
import io.hexlet.spring.repository.PostRepository;
import io.hexlet.spring.util.UserUtils;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/api")
public class PostsController {

    @Autowired
    private PostRepository repository;

    @Autowired
    private PostMapper postMapper;

    @GetMapping("/posts")
    @ResponseStatus(HttpStatus.OK)
    public List<PostDTO> index() {
        var posts = repository.findAll();
        var result = posts.stream()
            .map(postMapper::map)
            .toList();

        return result;
    }

    @PostMapping("/posts")
    @ResponseStatus(HttpStatus.CREATED)
    public PostDTO create(@Valid @RequestBody PostCreateDTO postData) {
        var post = postMapper.map(postData);
        repository.save(post);
        var postDTO = postMapper.map(post);
        return postDTO;
    }
}
```

```

@GetMapping("/posts/{id}")
@ResponseStatus(HttpStatus.OK)
public PostDTO show(@PathVariable Long id) {
    var post = repository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Not Found: " + id))
    var postDTO = postMapper.map(post);
    return postDTO;
}

@PutMapping("/posts/{id}")
@ResponseStatus(HttpStatus.OK)
public PostDTO update(@RequestBody @Valid PostUpdatedDTO postData, @PathVariable
    var post = repository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Not Found: " + id))
    postMapper.update(postData, post);
    repository.save(post);
    var postDTO = postMapper.map(post);
    return postDTO;
}

@DeleteMapping("/posts/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void delete(@PathVariable Long id) {
    repository.deleteById(id);
}
}

```

В целом, CRUD не ограничивается только перечисленными методами. По ситуации методов может быть больше.

Что такое Lombok

[Проект Lombok](#) (далее – просто *Lombok*) – это основанная на аннотациях библиотека Java, позволяющая сократить шаблонный код. В Lombok предлагаются различные аннотации, цель которых – заменить ненужный повторяющийся код, писать который утомительно. Например, Lombok избавит вас от написания безаргументных конструкторов, методов `toString()`, `equals()` и `hashCode()`, для этого нужно просто добавить несколько аннотаций. Плюс, как будет показано ниже, эта библиотека подключается прямо к вашей IDE, поэтому работа воспринимается так, словно вы написали весь шаблонный код сами.

Установить Lombok легко – добавьте `lombok` к вашим зависимостям. Если вы пользуетесь [Gradle](#), допишите следующие две строки с зависимостями в ваш файл `build.gradle`:

```
compileOnly 'org.projectlombok:lombok:1.18.20'

annotationProcessor 'org.projectlombok:lombok:1.18.20'
```

Тогда как если работаете с [Maven](#), добавьте следующую зависимость в ваш файл `pom.xml`:

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.18.20</version>
  <scope>provided</scope>
</dependency>
```

Кроме того, добавьте зависимость Lombok в конфигурационный раздел `maven-compiler-plugin` следующим образом:

```
<build>
  <plugins>

    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.5.1</version>
      <configuration>
        <source>11</source> <!-- depending on your
```



```

project -->
    <target>11</target> <!-- depending on your
project -->
    <annotationProcessorPaths>
        <path>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <version>1.18.20</version>
        </path>
    </annotationProcessorPaths>
</configuration>
</plugin>

<!-- ... -->

</plugins>
</build>

```

Теперь вы настроили все необходимое, чтобы приступить к работе с Lombok.

Наиболее распространенные аннотации Lombok

Ниже перечислены самые распространенные и важные аннотации Lombok. Каждая из них будет объяснена и рассмотрена на практике в сравнении с эквивалентным «переводом» на обычный язык Java. Чтобы посмотреть примеры и получить дополнительную поддержку, можно щелкнуть по любой аннотации и посетить посвященную ей страницу на сайте с [официальной документацией по Lombok](#).

@Getter, @Setter

Когда после аннотировано @Getter и/или @Setter, Lombok автоматически сгенерирует заданный по умолчанию геттер и/или сеттер, соответственно. Заданная по умолчанию реализация геттеров просто берет на себя возврат аннотированного поля. Аналогично, заданная по умолчанию реализация сеттеров принимает один параметр того же типа, что и аннотированное поле, и просто устанавливает его в полученное значение. Когда поле под названием value аннотируется одновременно @Getter и @Setter, Lombok определит метод getValue() (или isValue(), если это поле типа boolean) и setValue(). Сгенерированный метод, действующий как геттер и сеттер, будет публичным (public), если не указан конкретный уровень доступа

(AccessLevel). Допустимые значения AccessLevel – это PUBLIC, PROTECTED, PACKAGE и PRIVATE. Обратите внимание: можно аннотировать и целый класс. В таком случае данная логика будет применяться к каждому из его полей.

C Lombok

@Getter

@Setter

```
public class Author {  
    private int id;  
    private String name;  
    @Setter(AccessLevel.PROTECTED)  
    private String surname;  
}
```

Просто Java

```
public class User {  
    private int id;  
    private String name;  
    private String surname;  
  
    public int getId() {  
        return id;  
    }  
  
    public void setId(int id) {  
        this.id = id;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public String getSurname() {  
        return surname;  
    }  
  
    protected void setSurname(String surname) {  
        this.surname = surname;  
    }  
}
```

@NoArgsConstructor, @RequiredArgsConstructor, @AllArgsConstructor

Когда класс аннотирован @NoArgsConstructor, Lombok возьмет на себя автоматическую генерацию непараметризованного конструктора. Аналогично, когда класс аннотирован @AllArgsConstructor, будет сгенерирован конструктор с одним параметром для каждого поля вашего класса. Далее, @RequiredArgsConstructor позволит получить конструктор с параметром для каждого поля, но эти параметры потребуют специальной обработки. В частности, здесь мы будем работать с неинициализированными полями final, а также с любыми полями, помеченными как @NotNull, которые не инициализируются при объявлении. Пожалуйста, не забудьте, что статические поля будут этими аннотациями игнорироваться.

С Lombok

```
@NoArgsConstructor
@AllArgsConstructor
@RequiredArgsConstructor
public class Author {
    private int id;
    private String name;
    private String surname;
    private final String birthPlace;
}
```

Просто Java

```
public class Author {
    private int id;
    private String name;
    private String surname;
    private final String birthPlace;

    // @NoArgsConstructor
    public Author() {}

    // @AllArgsConstructor
    public Author(int id, String name, String surname, String birthPlace)
    {
        this.id = id
        this.name = name
        this.surname = surname
        this.birthPlace = birthPlace
    }
}
```

```

    // @RequiredArgsConstructor
    public Author(String birthPlace) {
        this.birthPlace = birthPlace
    }
}

```

@ToString

Когда класс аннотирован `@ToString`, Lombok позаботится о том, чтобы сгенерировать нужную реализацию метода `toString()`. По умолчанию будет возвращаться строка, в которой содержится имя класса, за которой следуют значения всех полей, разделенные запятыми. Устанавливая параметр `includeFieldNames` в значение *true*, добьемся того, что имя каждого поля будет ставиться перед его значением. По умолчанию при генерировании метода `toString()` будут учитываться все нестатические поля. Если мы хотим, чтобы Lombok проигнорировал поле, нужно аннотировать его `@ToString.Exclude`. Как вариант, можете сами указать, какие поля на ваше усмотрение должны аннотироваться, для этого воспользуйтесь `@ToString(onlyExplicitlyIncluded = true)`. Затем пометьте при помощи `@ToString.Include` каждое поле, которое хотите включить.

C Lombok

```

@ToString(includeFieldNames=true)
public class Author {
    private int id;
    private String name;
    private String surname;
}

```

Просто Java

```

public class Author {
    private int id;
    private String name;
    private String surname;

    @Override
    public String toString() {
        return "Author(id=" + this.id + ", name=" + this.name + ",
surname=" + this.surname + ")";
    }
}

```

@EqualsAndHashCode

Если аннотировать класс при помощи `@EqualsAndHashCode`, то Lombok автоматически реализует за вас методы `equals()` и `hashCode()`. По умолчанию будут учитываться все нестатические непереходные поля. Можно повлиять на то, какие поля будут использоваться, аннотировав их `@EqualsAndHashCode.Include` или `@EqualsAndHashCode.Exclude`. Как вариант, можете аннотировать ваш класс при помощи `@EqualsAndHashCode(onlyExplicitlyIncluded = true)`, а затем в точности указать, какие поля или методы вы хотите использовать, пометив их `@EqualsAndHashCode.Include`. Обратите внимание: Lombok сгенерирует методы `equals()` и `hashCode()`, не нарушая контракта между ними. Перейдите по ссылкам о двух этих методах на сайт с официальной документацией Java и почитайте подробнее о тех контрактах, которым должны удовлетворять реализации `equals()` и `hashCode()`.

С Lombok

@Getter

@Setter

@EqualsAndHashCode

```
public class Author {  
    private int id;  
    private String name;  
    private String surname;  
}
```

Просто Java

```
public class Author {
```

```
    // геттеры и сеттеры ...
```

```
@Override
```

```
public int hashCode() {  
    final int PRIME = 31;  
    int result = 1;  
    result = prime * result + id;  
    result = prime * result + ((name == null) ? 0 : name.hashCode());  
    result = prime * result + ((surname == null) ? 0 :  
surname.hashCode());  
    return result;  
}
```

```
@Override
```

```
public boolean equals(Object o) {  
    if (o == this) return true;
```

```

        if (!(o instanceof Author)) return false;
        Author other = (Author) o;
        if (!other.canEqual((Object)this)) return false;
        if (this.getId() == null ? other.getId() != null :
!this.getId().equals(other.getId())) return false;
        if (this.getName() == null ? other.getName() != null :
!this.getName().equals(other.getName())) return false;
        if (this.getSurname() == null ? other.getSurname() != null :
!this.getSurname().equals(other.getSurname())) return false;
        return true;
    }
}

```

@NonNull

Можно аннотировать при помощи @NonNull компонент записи, параметр, метод или целый конструктор, В таком случае Lombok сгенерирует для вас команды для проверки на ноль.

C Lombok

```

public class Author {
    private int id;
    private String name;
    private String surname;

    public Author(
        @NonNull int id,
        @NonNull String name,
        String surname
    ) {
        this.id = id;
        this.name = name;
        this.surname = surname;
    }
}

```

Просто Java

```

public class Author {
    private int id;
    private String name;
    private String surname;

    public Author(
        int id,

```

```

        String name,
        String surname
    ) {
        if (id == null) {
            throw new NullPointerException("id is marked @NonNull but is
null");
        }
        this.id = id;
        if (name == null) {
            throw new NullPointerException("name is marked @NonNull but is
null");
        }
        this.name = name;
        this.surname = surname;
    }
}

```

@Data

@Data – это сокращенная аннотация, сочетающая возможности @ToString, @EqualsAndHashCode, @Getter @Setter и @RequiredArgsConstructor. Так что @Data генерирует весь шаблонный код, вовлеченный в работу с объектами POJO ([Plain Old Java Objects](#)). Это, в частности, дает нам геттеры для всех полей, сеттеры для всех нефинальных полей, правильные реализации toString, equals и hashCode, охватывающие все поля класса, а также конструктор для всех финальных полей.

C Lombok

```

@Data
public class Author {
    private final int id;
    private String name;
    private String surname;
}

```

Просто Java

```

public class Author {
    private final int id;
    private String name;
    private String surname;

    public Author(int id) {
        this.id = id;
    }
}

```

```

public int getId() {
    return id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getSurname() {
    return surname;
}

public void setSurname(String surname) {
    this.surname = surname;
}

@Override
public int hashCode() {
    final int PRIME = 31;
    int result = 1;
    result = prime * result + getId();
    result = prime * result + ((getName() == null) ? 0 :
getName().hashCode());
    result = prime * result + ((getSurname() == null) ? 0 :
getSurname().hashCode());
    return result;
}

@Override
public boolean equals(Object o) {
    if (o == this) return true;
    if (!(o instanceof Author)) return false;
    Author other = (Author) o;
    if (!other.canEqual((Object)this)) return false;
    if (this.getId() == null ? other.getId() != null :
!this.getId().equals(other.getId())) return false;
    if (this.getName() == null ? other.getName() != null :
!this.getName().equals(other.getName())) return false;
    if (this.getSurname() == null ? other.getSurname() != null :
!this.getSurname().equals(other.getSurname())) return false;
    return true;
}

```



```
}
```

@Value

@Value – это неизменяемый вариант @Data. С этой аннотацией Lombok по умолчанию делает все поля private и final. Кроме того, сеттеры не генерируются, а класс как таковой помечается final. Таким образом, от этого класса нельзя наследовать. Точно как и в случае с @Data, создаются реализации toString(), equals() и hashCode().

С Lombok

@Data

```
public class Author {  
    int id;  
    String name;  
    String surname;  
}
```

Просто Java

```
public final class Author {  
    private final int id;  
    private final String name;  
    private final String surname;  
  
    public Author(int id, String name, String surname) {  
        this.id = id  
        this.name = name  
        this.surname = surname  
    }  
  
    public int getId() {  
        return id;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public String getSurname() {  
        return surname;  
    }  
  
    @Override  
    public int hashCode() {
```

```

        final int PRIME = 31;
        int result = 1;
        result = prime * result + getId();
        result = prime * result + ((getName() == null) ? 0 :
getName().hashCode());
        result = prime * result + ((getSurname() == null) ? 0 :
getSurname().hashCode());
        return result;
    }

    @Override
    public boolean equals(Object o) {
        if (o == this) return true;
        if (!(o instanceof Author)) return false;
        Author other = (Author) o;
        if (!other.canEqual((Object)this)) return false;
        if (this.getId() == null ? other.getId() != null :
!this.getId().equals(other.getId())) return false;
        if (this.getName() == null ? other.getName() != null :
!this.getName().equals(other.getName())) return false;
        if (this.getSurname() == null ? other.getSurname() != null :
!this.getSurname().equals(other.getSurname())) return false;
        return true;
    }
}

```